CrewAI Code Analysis Tools

Technical Architecture & Agent Tooling Reference

A comprehensive guide to the 4 specialized tool modules powering the AI-driven code analysis system

Document Type	Technical Reference
Framework	CrewAI + Python AST
Total Tools	21 specialized tools
Agent Roles	4 specialized agents

Table of Contents

1. ast_tools.py — The Code Structure Inspector

5 tools for AST parsing and code smell detection

2. code_analysis_tools.py — The Architectural Surveyor

4 tools for file structure and architecture analysis

3. dependency_graph_tools.py — The Coupling & Cycle Analyst

5 tools for dependency mapping and stability metrics

4. git_tools.py — The Change History Detective

7 tools for version control and ownership analysis

5. How the 4 Files Work Together

Agent orchestration and data flow

1. ast_tools.py

The Code Structure Inspector

This module serves as the **deep code parser** for the CrewAI system. It reads Python source files and converts them into Abstract Syntax Trees (AST), then walks through every node to extract functions, classes, detect anti-patterns, and map call relationships.

What It Does

Uses Python's built-in **ast** module to parse `.py` files into structured node trees. It then traverses these trees to count and classify every structural element — functions, classes, imports, control-flow statements — while calculating complexity metrics that indicate where code is hardest to maintain.

How It Works

- **AST Parsing:** Each Python file is read and passed to `ast.parse()`, producing a tree of syntax nodes.
- **Node Walking:** `ast.walk()` recursively visits every node, counting occurrences of each node type (FunctionDef, ClassDef, If, For, etc.).
- **Complexity Calculation:** Cyclomatic complexity is computed by counting decision points (if, while, for, except, boolean operators). Cognitive complexity weights nesting depth.
- **Code Smell Detection:** Specific AST patterns trigger alerts — `ast.ExceptHandler` with `type=None` flags bare excepts; mutable defaults (list/dict/set) in function arguments are caught.
- **Call Graph Building:** Tracks `ast.Call` nodes within each function scope to build function-level dependency maps and identify dead code.

Available Tools (5)

Tool Name	Purpose	Output
parse_ast_tree	Parse single file to AST	Node counts, tree dump, parse status
extract_functions	Extract all functions/methods	Complexity scores, line counts, async flags
extract_classes	Extract class definitions	Inheritance tree, composition, god-class detection
find_code_smells	Detect anti-patterns	Bare excepts, mutable defaults, deep nesting, TODOs
get_function_dependencies	Function call analysis	Call graphs, dead code candidates, entry points

Key Insight: This is the only module that performs true static analysis at the syntax level. All other modules rely on its outputs (function lists, class hierarchies) for higher-level reasoning.

2. code_analysis_tools.py

The Architectural Surveyor

This module acts as the **high-level architect** of the system. It analyzes the physical organization of the codebase — file structure, module boundaries, language distribution, and architectural layering — to answer the fundamental question: *Is this codebase well-organized?*

What It Does

Recursively walks the directory tree, skipping non-code artifacts, and categorizes every file by language and module. It then applies heuristics to classify modules into architectural roles (API, Data, Core, Utility, Service) and detects violations of clean architecture principles.

How It Works

- **Directory Traversal:** Uses `os.walk()` with filters to skip `.git`, `node_modules`, `__pycache__`, virtual environments, and build artifacts.
- **Language Detection:** Maps file extensions to languages — .py → Python, .js/.ts → JavaScript/TypeScript, .go → Go, .rs → Rust, etc.
- **Line Classification:** For each file, counts code lines vs. comment lines vs. blank lines to compute comment coverage ratios.
- **Module Classification:** Groups files by directory path into modules, then applies naming heuristics: names containing 'api'/'controller' → API layer; 'model'/'repository' → Data layer; high incoming dependencies → Core.
- **Architecture Violation Detection:** Flags layer violations (Data importing API), god modules (>15 files or 8000 lines), and missing test coverage.
- **Quality Scoring:** Computes an A–F grade based on comment ratio, duplication, complexity, and file size distribution.

Available Tools (4)

Tool Name	Purpose	Output
analyze_file_structure	Directory tree analysis	File inventory, language distribution, size metrics
identify_modules	Module boundary detection	Architectural roles, debt hotspots, instability
calculate_code_metrics	Quality metrics	LOC breakdown, duplication, complexity, A–F score
detect_architectural_issues	Layer violation detection	God modules, layer violations, test gaps

Key Insight: This module bridges the gap between raw AST data and architectural reasoning. It transforms syntax-level facts into module-level judgments that architects and tech leads care about.

3. dependency_graph_tools.py

The Coupling & Cycle Analyst

This module is the **relationship mapper** of the system. It quantifies *who depends on whom* across the codebase, detects circular dependencies, and evaluates each module's stability using established software architecture metrics.

What It Does

Scans all Python files to extract import statements, categorizes them as internal, standard library, or external, then builds a complete dependency graph. It applies Robert C. Martin's stability metrics to classify modules into architectural zones: Main Sequence, Zone of Pain, or Zone of Uselessness.

How It Works

- **Two-Pass Scan:** First pass collects all internal module names. Second pass parses AST import nodes to classify each import.
- **Import Categorization:** Compares import base names against a stdlib whitelist and the internal module set. Anything else is external.
- **Coupling Calculation:** Counts cross-module imports and normalizes to a 0–1 strength scale. Tracks bidirectional (mutual) dependencies.
- **Graph Generation:** Produces nodes/edges output compatible with Vis.js, D3, or Cytoscape for interactive visualization.
- **Cycle Detection (DFS):** Uses Depth-First Search to find circular dependencies. Tracks the current path; revisiting a node on the path indicates a cycle. Normalizes cycles to avoid duplicate reporting.
- **Stability Metrics:** Instability = outgoing / (incoming + outgoing). Abstractness is estimated from naming heuristics. Distance from Main Sequence = |A + I – 1|.

Available Tools (5)

Tool Name	Purpose	Output
extract_imports	Import extraction	Internal/external/stdlib dependency lists
calculate_coupling_score	Coupling analysis	0–1 strength scores, bidirectional flags
build_dependency_graph	Graph generation	Nodes/edges for visualization, centrality scores
detect_circular_dependencies	Cycle detection	DFS cycles, length distribution, problematic modules
analyze_module_stability	Stability metrics	Zone classification, main sequence distance

Key Insight: Circular dependencies are the #1 cause of brittle architectures. This module's DFS cycle detector finds them before they become deployment blockers.

4. git_tools.py

The Change History Detective

This module is the **historian** of the system. It mines the Git repository's commit history to find which files change most often, who owns what code, where knowledge is concentrated, and which files are likely refactoring targets based on bug-fix patterns.

What It Does

Executes `git` commands via Python's `subprocess` module to extract commit metadata, file blame data, and change statistics. It correlates commit messages with file modifications to surface process issues and technical debt indicators.

How It Works

- **Git Command Execution:** Runs git commands with timeout protection and error handling for non-git directories.
- **Commit History Parsing:** Uses custom `--pretty=format` strings to parse commit metadata (hash, author, date, message, stats) efficiently.
- **File Blame Analysis:** Runs `git blame --line-porcelain` per file to get line-by-line authorship and age. Calculates bus factor (how many authors own >10%).
- **Churn Detection:** Counts how many times each file appears across commits. Churn score = commits × author diversity. High churn = instability.
- **Commit Pattern Analysis:** Detects conventional commit compliance (feat:/fix:/refactor:), WIP commits, revert frequency, and long gaps in activity.
- **Ownership Mapping:** Computes code ownership across all tracked files using blame data. Calculates Herfindahl index for knowledge concentration risk.
- **Refactoring Prioritization:** Correlates commit messages with file changes to find files with high bug-fix ratios — files needing frequent fixes are poorly designed.

Available Tools (7)

Tool Name	Purpose	Output
get_git_history	Commit log retrieval	Author stats, velocity, daily distribution
get_file_blame	Line ownership	Per-line authorship, bus factor, code age
detect_churn	Change frequency	Hotspots, churn score, risk levels
get_recent_changes	Recent activity	Impact scores, refactoring detection
analyze_commit_patterns	Process quality	Conventional commits, bus factor, quality score
get_code_ownership	Knowledge map	Author domains, orphan files, concentration
detect_refactoring_candidates	Priority ranking	Bug-fix ratio, priority scores

Key Insight: Code that changes frequently is code that hurts. This module's churn analysis connects version control history to technical debt in a way static analysis cannot.

5. How the 4 Files Work Together

The four modules form a **layered analysis pipeline**, where each layer builds upon the previous one. They are designed to be assigned to separate CrewAI agents that collaborate through a shared analysis cache.

Agent Orchestration

Agent Role	File	Input	Output
Parser	ast_tools.py	Raw .py source	Functions, classes, smells, complexity
Architect	code_analysis_tools.py	Directory tree	Module map, layer violations, quality score
Dependency Analyst	dependency_graph_tools.py	Import statements	Coupling matrix, cycles, stability zones
Historian	git_tools.py	Git commit log	Churn hotspots, ownership, refactoring targets

Data Flow & Caching

Each module maintains its own `_analysis_cache` dictionary. When agents run sequentially, subsequent agents can reuse previously computed data without re-parsing:

> **Example flow:** The Dependency Analyst runs `extract_imports()` and caches the result. Later, the Architect's `identify_modules()` checks `_analysis_cache.get('imports')` — if present, it reuses it. This avoids redundant AST parsing and keeps the multi-agent system efficient.

Integration Example

A typical CrewAI crew configuration using all 4 modules:

```
from crewai import Agent, Task, Crew
from ast_tools import extract_functions, find_code_smells
from code_analysis_tools import analyze_file_structure, detect_architectural_issues
from dependency_graph_tools import detect_circular_dependencies, calculate_coupling_score
from git_tools import detect_churn, get_code_ownership

# Create specialized agents
ast_agent = Agent(
    role='AST Analyzer',
    goal='Find complex functions and code smells',
    tools=[extract_functions, find_code_smells]
)

arch_agent = Agent(
    role='Architect',
    goal='Detect layering violations and module issues',
    tools=[analyze_file_structure, detect_architectural_issues]
)

dep_agent = Agent(
```

```
    role='Dependency Analyst',
    goal='Find circular dependencies and coupling issues',
    tools=[detect_circular_dependencies, calculate_coupling_score]
)

git_agent = Agent(
    role='Git Historian',
    goal='Identify high-churn files and ownership gaps',
    tools=[detect_churn, get_code_ownership]
)
```

Summary Statistics

Metric	Value
Total Specialized Tools	21
Total Lines of Code	~2,275
Agent Roles	4 (Parser, Architect, Dependency Analyst, Historian)
Analysis Layers	4 (Syntax → Structure → Dependencies → History)
Cache Strategy	Per-module _analysis_cache with cross-module reuse
Supported Languages	Python (primary), JS/TS, Go, Rust, Java, C/C++, Ruby, PHP, Swift, Kotlin

This document was generated as a technical reference for the CrewAI multi-agent code analysis system. Each module is designed to operate independently while contributing to a unified analysis pipeline that detects technical debt, architectural issues, and refactoring opportunities in software repositories.